\

PROGRAM:2.1

2.1 OBJECTIVE: Program to simulate FCFS CPU scheduling Algorithm.

2.2 THEORY:

First-Come-First-Served algorithm is the simplest scheduling algorithm. Processes are dispatched according to their arrival time on the ready queue. Being a non preemptive discipline, once a process has a CPU, it runs to completion. The FCFS scheduling is fair in the formal sense or human sense of fairness but it is unfair in the sense that long jobs make short jobs wait and unimportant jobs make important jobs wait.

FCFS is more predictable than most of other schemes since it offers time. FCFS scheme is not useful in scheduling interactive users because it cannot guarantee good response time. The code for FCFS scheduling is simple to write and understand. One of the major drawback of this scheme is that the average time is often quite long. The First-Come-First-Served algorithm is rarely used as a master scheme in modern operating systems but it is often embedded within other schemes. It is a non preemption type scheduling algorithm.

2.3 SOURCE CODE:

```
#include<stdio.h>
int main()
{
   int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;
   printf("Enter total number of processes(maximum 20): ");
   scanf("%d",&n);
   printf("\nEnter Process Burst Time:\n");
   for(i=0;i<n;i++)
   {
      printf("P[%d]: ",i+1);
      scanf("%d",&bt[i]);
   }
   wt[0]=0;
   for(i=1;i<n;i++)
   {
      wt[i]=0;
      for(j=0;j<i;j++)
      wt[i]+=bt[j];
   }

   printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time");
   //calculating turnaround time
   for(i=0;i<n;i++)
   {
      tat[i]=bt[i]+wt[i];
      avwt+=wt[i];
      avtat+=tat[i];
      printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
   }
   avwt/=i;
   avtat/=i;
```

```
    printf("\n\nAverage Waiting Time:%d",avwt);
    printf("\nAverage Turnaround Time:%d",avtat);
    return 0;
}
```

2.4 OUTPUT:

```
test_os.cpp
C:\Users\hp\Documents\dev c++\test_os.exe
Enter total number of processes(maximum 20): 4

Enter Process Burst Time:
P[1]: 14
P[2]: 7
P[3]: 3
P[4]: 9

Process         Burst Time      Waiting Time    Turnaround Time
P[1]            14              0               14
P[2]            7               14              21
P[3]            3               21              24
P[4]            9               24              33

Average Waiting Time:14
Average Turnaround Time:23
--------------------------------
Process exited after 13.83 seconds with return value 0
Press any key to continue . . .
```

PROGRAM: 2.2

3.1 OBJECTIVE: Program to simulate SJF CPU scheduling Algorithm.

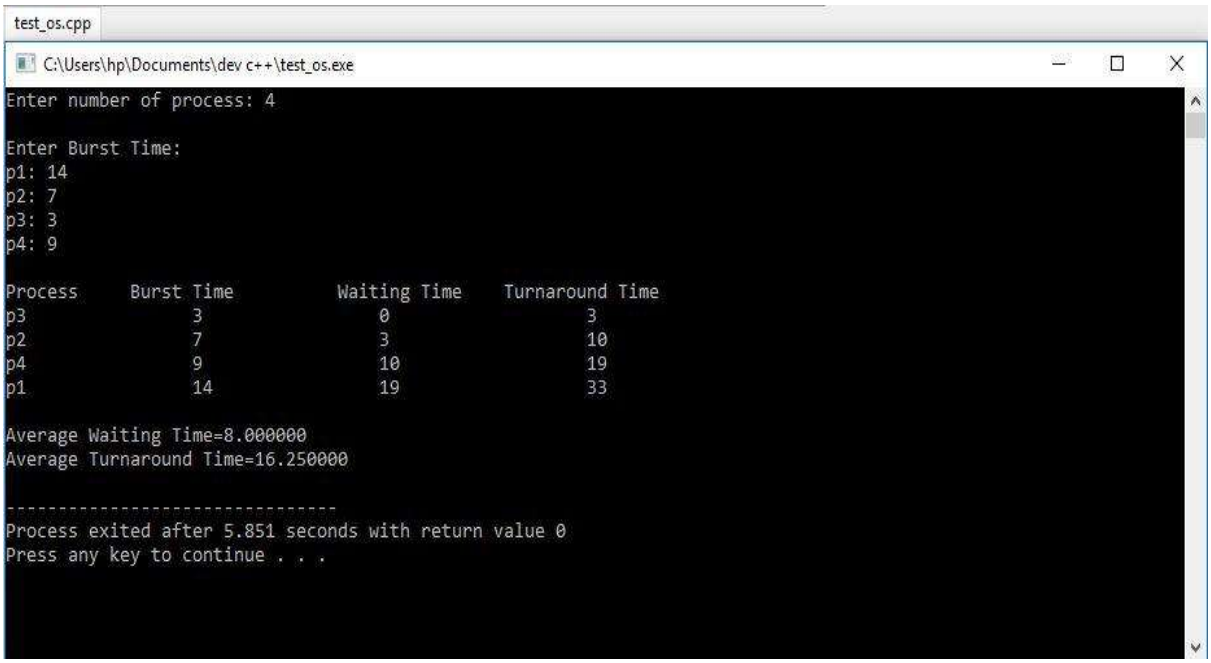
3.2 THEORY:

Shortest-Job-First (SJF) is a non-preemptive discipline in which waiting job (or process) with the smallest estimated run-time-to-completion is run next. In other words, when CPU is available, it is assigned to the process that has smallest next CPU burst.

The SJF scheduling is especially appropriate for batch jobs for which the run times are known in advance. Since the SJF scheduling algorithm gives the minimum average time for a given set of processes, it is probably optimal. The SJF algorithm favors short jobs (or processors) at the expense of longer ones. The obvious problem with SJF scheme is that it requires precise knowledge of how long a job or process will run, and this information is not usually available. The best SJF algorithm can do is to rely on user estimates of run times.

3.3 SOURCE CODE

```
#include<stdio.h>
int main()
{
    int bt[20],p[20],wt[20],tat[20],i,j,n,total=0,pos,temp;
    float avg_wt,avg_tat;
    printf("Enter number of process: ");
    scanf("%d",&n);
    printf("\nEnter Burst Time:\n");
    for(i=0;i<n;i++)
    {
        printf("p%d: ",i+1);
        scanf("%d",&bt[i]);
        p[i]=i+1;
```

```
    }
    for(i=0;i<n;i++)
    {   pos=i;
        for(j=i+1;j<n;j++)
        {       if(bt[j]<bt[pos])
                pos=j;
        }
        temp=bt[i];
        bt[i]=bt[pos];
        bt[pos]=temp;
        temp=p[i];
        p[i]=p[pos];
        p[pos]=temp;
    }
    wt[0]=0;
    for(i=1;i<n;i++)
    {   wt[i]=0;
        for(j=0;j<i;j++)
        wt[i]+=bt[j];
         total+=wt[i];
    }
    avg_wt=(float)total/n;
    total=0;
    printf("\nProcess\t    Burst Time    \tWaiting Time\tTurnaround Time");
    for(i=0;i<n;i++)
    {  tat[i]=bt[i]+wt[i];
        total+=tat[i];
        printf("\np%d\t\t  %d\t\t    %d\t\t\t%d",p[i],bt[i],wt[i],tat[i]);
    }
   avg_tat=(float)total/n;
    printf("\n\nAverage Waiting Time=%f",avg_wt);
    printf("\nAverage Turnaround Time=%f\n",avg_tat);
    return 0;
}
```

3.4 OUTPUT: